Optimization

Optimization involves executing a function on a set of various configurations with an aim to optimize the performance of a strategy, and/or to optimize the CPU or RAM performance of a pipeline.

Question

Learn more in Pairs trading tutorial.

Parameterization

The first and easiest approach revolves around testing a single parameter combination at a time, which utilizes as little RAM as possible but may take longer to run if the function isn't written in pure Numba and has a fixed overhead (e.g., conversion from Pandas to NumPy and back) that adds to the total execution time with each run. For this, create a pipeline function that runs a set of single values and decorate it with `@vbt.parameterized`. To test multiple parameters, wrap each parameter argument with **Param**.

Example

See an example in Parameterized decorator.

Decoration

To parameterize any function, we have to decorate (or wrap) it with `@vbt.parameterized`. This will return a new function with the same name and arguments as the original one. The only difference: this new function will process passed arguments, build parameter combinations, call the original function on each parameter combination, and merge the results of all combinations.

Process only one parameter combination at a time

```
@vbt.parameterized
def my_pipeline(data, fast_window, slow_window):  (1)
    ...
    return result  (2)

results = my_pipeline(  (3)
    data,
    vbt.Param(fast_windows),  (4)
    vbt.Param(slow_windows)
)
```

1. Arguments can be anything. Here we're expecting a data instance and two parameters: `fast_window` and `slow_window`, they will be passed by the decorator as single values.
2. Do some calculations on the received parameter combination and return a result, which can be anything
3. Run the function the same way as without the decorator
4. Wrap multiple values with `vbt.Param` under each parameter

To keep the original function separate from the decorated one, we can decorate it after it has been defined and give the decorated function another name.

Decorate a function later

```
def my_pipeline(data, fast_window, slow_window):
    ...
    return result

my_param_pipeline = vbt.parameterized(my_pipeline)
results = my_param_pipeline(...)
```

Merging

The code above returns a list of results, one per parameter combination. To return the grid of parameter combinations as well, pass `return_param_index=True` to the decorator. Alternatively, let VBT merge the results into one or more Pandas objects and attach the grid to their index or columns by specifying the merging function (see **resolve_merge_func**).

Various merging configurations

```
@vbt.parameterized(return_param_index=True)  (1)
def my_pipeline(...):
    ...
    return result

results, param_index = my_pipeline(...)

# ____________________________________________________________
```

```
@vbt.parameterized(merge_func="concat")  ❷
def my_pipeline(...):
    ...
    return pf.sharpe_ratio

sharpe_ratio = my_pipeline(...)

# ____________________________________________________________

@vbt.parameterized(merge_func="concat")
def my_pipeline(...):
    ...
    return pf.sharpe_ratio, pf.win_rate

sharpe_ratio, win_rate = my_pipeline(...)

# ____________________________________________________________

@vbt.parameterized(merge_func="column_stack")  ❸
def my_pipeline(...):
    ...
    return entries, exits

entries, exits = my_pipeline(...)

# ____________________________________________________________

@vbt.parameterized(merge_func="row_stack")  ❹
def my_pipeline(...):
    ...
    return pf.value

value = my_pipeline(...)

# ____________________________________________________________

@vbt.parameterized(merge_func=("concat", "column_stack"))  ❺
def my_pipeline(...):
    ...
    return pf.sharpe_ratio, pf.value

sharpe_ratio, value = my_pipeline(...)

# ____________________________________________________________

def merge_func(results, param_index):
    return pd.Series(results, index=param_index)

@vbt.parameterized(
    merge_func=merge_func,  ❻
    merge_kwargs=dict(param_index=vbt.Rep("param_index"))  ❼
)
def my_pipeline(...):
    ...
    return pf.sharpe_ratio

sharpe_ratio = my_pipeline(...)
```

❶ Return the results along with the parameter grid

❷ If the function returns a single number (or a tuple of such), concatenate all numbers into a Series with parameter combinations as index. Useful for returning metrics such as Sharpe ratio.

❸ If the function returns an array (or a tuple of such), stack all arrays along columns into a DataFrame with parameter combinations as an outermost column level. Useful for returning indicator arrays.

❹ If the function returns an array (or a tuple of such), stack all arrays along rows into a Series/DataFrame with parameter combinations as an outermost index level. Useful for cross-validation.

❺ If the function returns a number and an array, return a Series of concatenated numbers and a DataFrame of arrays stacked along columns

❻ Pass a custom merging function

❼ Use an expression template to pass the parameter index as a keyword argument

We can also use annotations to specify the merging function(s).

```
@vbt.parameterized
def my_pipeline(...) -> "concat":  ❶
    ...
    return result

# ____________________________________________________________

@vbt.parameterized
def my_pipeline(...) -> ("concat", "column_stack"):  ❷
    ...
    return result1, result2

# ____________________________________________________________

@vbt.parameterized
def my_pipeline(...) -> (  ❸
```

```
    vbt.MergeFunc("concat", wrap=False),
    vbt.MergeFunc("column_stack", wrap=False)
):
    ...
    return result1, result2
```

1. Concatenate results
2. Concatenate instances of the first result and column-stack instances of the second result
3. Same as above but provide keyword arguments to each merging function

Generation

The grid of parameter combinations can be controlled by individual parameters. By default, vectorbtpro will build a Cartesian product of all parameters. To avoid building the product between some parameters, they can be assigned to the same product `level`. To filter out unwanted parameter configurations, specify the `condition` as a boolean expression where variables are parameter names. Such a condition will be evaluated on each parameter combination, and if it returns True, the combination will be kept. To change the appearance of a parameter in the parameter index, `keys` with human-readable strings can be provided. A parameter can also be hidden entirely by setting `hide=True`.

Various parameter configurations

```
sma_crossover(  # 1
    data=data,
    fast_window=vbt.Param(windows, condition="fast_window < slow_window"),
    slow_window=vbt.Param(windows),
)

# ----------------------------------------------------------------

sma_crossover(  # 2
    data=vbt.Param(data),
    fast_window=vbt.Param(windows, condition="fast_window < slow_window"),
    slow_window=vbt.Param(windows),
)

# ----------------------------------------------------------------

from itertools import combinations

fast_windows, slow_windows = zip(*combinations(windows, 2))  # 3
sma_crossover(
    data=vbt.Param(data, level=0),
    fast_window=vbt.Param(fast_windows, level=1),
    slow_window=vbt.Param(slow_windows, level=1),
)

# ----------------------------------------------------------------

bbands_indicator(  # 4
    data=data,
    timeperiod=vbt.Param(timeperiods, level=0),
    upper_threshold=vbt.Param(thresholds, level=1, keys=pd.Index(thresholds, name="threshold")),
    lower_threshold=vbt.Param(thresholds, level=1, hide=True),
    _random_subset=1_000  # 5
)
```

1. Build a product of fast and slow windows while removing those where the fast window is longer than the slow window (e.g., 20 and 50 is ok but 50 and 20 doesn't make sense)
2. Same as above but test only one symbol at a time
3. Same as above but build the window combinations manually. The window parameters are now on the same level and won't build another product.
4. Test two parameters: time periods and thresholds. The upper and lower thresholds should both share the same values and only one `threshold` level should be displayed in the parameter index. Also, select a random subset of 1000 parameter combinations.
5. Arguments that are normally passed to the decorator can be also passed to the function itself by prepending an underscore

Example

See an example in Conditional parameters.

Warning

Testing 6 parameters with only 10 values each would generate staggering 1 million parameter combinations, thus make sure that your grids are not too wide, otherwise the generation part alone will take forever to run. This warning doesn't apply when you use `random_subset` though; in this case, VBT won't build the full grid but select random combinations dynamically. See an example in Lazy parameter grids.

We can also use annotations to specify which arguments are parameters and their default configuration.

Calculate the SMA crossover for one parameter combination at a time

```
@vbt.parameterized
def sma_crossover(
    data,
```

```
    fast_window: vbt.Param(condition="fast_window < slow_window"),
    slow_window: vbt.Param,
) -> "column_stack":
    fast_sma = data.run("talib:sma", fast_window, unpack=True)
    slow_sma = data.run("talib:sma", slow_window, unpack=True)
    upper_crossover = fast_sma.vbt.crossed_above(slow_sma)
    lower_crossover = fast_sma.vbt.crossed_below(slow_sma)
    signals = upper_crossover | lower_crossover
    return signals

signals = sma_crossover(data, fast_windows, slow_windows)
```

Pre-generation

To get the generated parameter combinations before (or without) calling the `@vbt.parameterized` decorator, we can pass the same parameters to **combine_params**.

Pre-generate parameter combinations

```
param_product, param_index = vbt.combine_params(
    fast_window=vbt.Param(windows, condition="fast_window < slow_window"),
    slow_window=vbt.Param(windows)
)

# ____________________________________________________________

param_product = vbt.combine_params(
    fast_window=vbt.Param(windows, condition="fast_window < slow_window"),
    slow_window=vbt.Param(windows),
    build_index=False  (1)
)
```

(1) Don't build the index. Return only the parameter product.

Execution

Each parameter combination involves one call of the pipeline function. To perform multiple calls in parallel, pass a dictionary named `execute_kwargs` with keyword arguments that should be forwarded to the function **execute**, which takes care of chunking and executing the function calls.

Various execution configurations

```
@vbt.parameterized  (1)
def my_pipeline(...):
    ...

# ____________________________________________________________

@vbt.parameterized(execute_kwargs=dict(chunk_len="auto", engine="threadpool"))  (2)
@njit(nogil=True)
def my_pipeline(...):
    ...

# ____________________________________________________________

@vbt.parameterized(execute_kwargs=dict(n_chunks="auto", distribute="chunks", engine="pathos"))  (3)
def my_pipeline(...):
    ...

# ____________________________________________________________

@vbt.parameterized  (4)
@njit(nogil=True)
def my_pipeline(...):
    ...

my_pipeline(
    ...,
    _execute_kwargs=dict(chunk_len="auto", engine="threadpool")
)

# ____________________________________________________________

@vbt.parameterized(execute_kwargs=dict(show_progress=False))  (5)
@njit(nogil=True)
def my_pipeline(...):
    ...

my_pipeline(
    ...,
    _execute_kwargs=dict(chunk_len="auto", engine="threadpool")  (6)
)
my_pipeline(
    ...,
    _execute_kwargs=vbt.atomic_dict(chunk_len="auto", engine="threadpool")  (7)
)
```

(1) Execute parameter combinations serially

(2) Distribute parameter combinations into chunks of an optimal length, and execute all parameter combinations **within each chunk** in parallel with multithreading (i.e., one parameter combination per thread) while executing chunks themselves serially

3. Divide parameter combinations into an optimal number of chunks, and execute **all chunks** in parallel with multiprocessing (i.e., one chunk per process) while executing all parameter combinations within each chunk serially
4. Parallelization can be enabled/disabled sporadically by prepending an underscore to `execute_kwargs` and passing it directly to the function
5. If there's already `execute_kwargs` active in the decorator, they will be merged together. To avoid merging, wrap any of the dicts with `vbt.atomic_dict`.
6. `show_progress=False`
7. `show_progress=True` (default)

> **Note**
>
> Threads are easier and faster to spawn than processes. Also, to execute a function in its own process, all the passed inputs and parameters need to be serialized and then deserialized, which takes time. Thus, multithreading is preferred, but it requires the function to release the GIL, which means either compiling the function with Numba and setting the `nogil` flag to True, or using exclusively NumPy.
>
> If this isn't possible, use multiprocessing but make sure that the function either doesn't take large arrays, or that one parameter combination takes a considerable amount of time to run. Otherwise, you may find parallelization making the execution even slower.

To run a code before/after the entire processing or even before/after each individual chunk, **execute** offers a number of callbacks.

Clear cache and collect garbage once in 3 chunks

```
def post_chunk_func(chunk_idx, flush_every):
    if (chunk_idx + 1) % flush_every == 0:
        vbt.flush()

@vbt.parameterized(
    post_chunk_func=post_chunk_func,
    post_chunk_kwargs=dict(
        chunk_idx=vbt.Rep("chunk_idx", eval_id="post_chunk_kwargs"),
        flush_every=3
    ),
    chunk_len=10  (1)
)
def my_pipeline(...):
    ...
```

1. Put 10 calls into one chunk, that is, flush each 30 calls

> **Tip**
>
> This works not only with `@vbt.parameterized` but also with other functions that use **execute** with chunking!

Total or partial?

Often, you should make a decision whether your pipeline should be parameterized totally or partially. Total parameterization means running the entire pipeline on each parameter combination, which is the easiest but also the most suitable approach if you have parameters being applied across multiple components of the pipeline, and/or if you want to trade in faster processing for lower memory consumption.

Parameterize an entire MA crossover pipeline

```
@vbt.parameterized(merge_func="concat")
def ma_crossover_sharpe(data, fast_window, slow_window):
    fast_ma = data.run("vbt:ma", window=fast_window, hide_params=True)
    slow_ma = data.run("vbt:ma", window=slow_window, hide_params=True)
    entries = fast_ma.ma_crossed_above(slow_ma)
    exits = fast_ma.ma_crossed_below(slow_ma)
    pf = vbt.PF.from_signals(data, entries, exits)
    return pf.sharpe_ratio

ma_crossover_sharpe(
    data,
    vbt.Param(fast_windows, condition="fast_window < slow_window"),
    vbt.Param(slow_windows)
)
```

Partial parameterization, on the other hand, is appropriate if you have only a few components in the pipeline where parameters are being applied, and if the remaining components of the pipeline know how to work with the results from the parameterized components. This may lead to a faster execution but also a higher memory consumption.

Parameterize only the signal part of a MA crossover pipeline

```
@vbt.parameterized(merge_func="column_stack")
def ma_crossover_signals(data, fast_window, slow_window):
    fast_ma = data.run("vbt:ma", window=fast_window, hide_params=True)
    slow_ma = data.run("vbt:ma", window=slow_window, hide_params=True)
    entries = fast_ma.ma_crossed_above(slow_ma)
    exits = fast_ma.ma_crossed_below(slow_ma)
    return entries, exits

def ma_crossover_sharpe(data, fast_windows, slow_windows):
```

```
    entries, exits = ma_crossover_signals(data, fast_windows, slow_windows)  (1)
    pf = vbt.PF.from_signals(data, entries, exits)  (2)
    return pf.sharpe_ratio

ma_crossover_sharpe(
    data,
    vbt.Param(fast_windows, condition="fast_window < slow_window"),
    vbt.Param(slow_windows)
)
```

(1) Parameter combinations become columns in the entry and exit arrays

(2) Simulator knows how to handle these additional columns

Flat or nested?

Another decision you should make is whether to handle all parameters by one decorator (flat parameterization) or distribute parameters across multiple decorators to implement a specific parameter hierarchy (nested parameterization). The former approach should be used if you want to treat all of your parameters equally and put them into the same bucket for generation and processing. In this case, the order of the parameters in combinations is defined by the order the parameters are passed to the function. For example, while the values of the first parameter will be processed strictly from the first to the last value, the values of any other parameter will be rotated.

Process all parameters at the same time in a MA crossover pipeline

```
@vbt.parameterized(merge_func="concat")
def ma_crossover_sharpe(data, symbol, fast_window, slow_window):
    symbol_data = data.select(symbol)  (1)
    fast_ma = symbol_data.run("vbt:ma", window=fast_window, hide_params=True)
    slow_ma = symbol_data.run("vbt:ma", window=slow_window, hide_params=True)
    entries = fast_ma.ma_crossed_above(slow_ma)
    exits = fast_ma.ma_crossed_below(slow_ma)
    pf = vbt.PF.from_signals(symbol_data, entries, exits)
    return pf.sharpe_ratio

ma_crossover_sharpe(
    data,
    vbt.Param(data.symbols),
    vbt.Param(fast_windows, condition="fast_window < slow_window"),
    vbt.Param(slow_windows),
)
```

(1) Symbol selection depends on the symbol only but is run for each combination of `symbol`, `fast_window`, and `slow_window` - unnecessary often!

The latter approach should be used if you want to define your own custom parameter hierarchy. For example, you may want to execute (such as parallelize) certain parameters differently, or you may want to reduce the number of invocations of certain parameters, or you may want to introduce special preprocessing and/or postprocessing to certain parameters.

First process symbols and then windows in a MA crossover pipeline

```
@vbt.parameterized(merge_func="concat", eval_id="inner")  (1)
def symbol_ma_crossover_sharpe(symbol_data, fast_window, slow_window):
    fast_ma = symbol_data.run("vbt:ma", window=fast_window, hide_params=True)
    slow_ma = symbol_data.run("vbt:ma", window=slow_window, hide_params=True)
    entries = fast_ma.ma_crossed_above(slow_ma)
    exits = fast_ma.ma_crossed_below(slow_ma)
    pf = vbt.PF.from_signals(symbol_data, entries, exits)
    return pf.sharpe_ratio

@vbt.parameterized(merge_func="concat", eval_id="outer")  (2)
def ma_crossover_sharpe(data, symbol, fast_windows, slow_windows):
    symbol_data = data.select(symbol)  (3)
    return symbol_ma_crossover_sharpe(symbol_data, fast_windows, slow_windows)  (4)

ma_crossover_sharpe(  (5)
    data,
    vbt.Param(data.symbols, eval_id="outer"),
    vbt.Param(fast_windows, eval_id="inner", condition="fast_window < slow_window"),
    vbt.Param(slow_windows, eval_id="inner")
)

# ------------------------------------------------------------

@vbt.parameterized(merge_func="concat", eval_id="outer")
@vbt.parameterized(merge_func="concat", eval_id="inner")
def ma_crossover_sharpe(data, fast_window, slow_window):  (6)
    fast_ma = data.run("vbt:ma", window=fast_window, hide_params=True)
    slow_ma = data.run("vbt:ma", window=slow_window, hide_params=True)
    entries = fast_ma.ma_crossed_above(slow_ma)
    exits = fast_ma.ma_crossed_below(slow_ma)
    pf = vbt.PF.from_signals(data, entries, exits)
    return pf.sharpe_ratio

ma_crossover_sharpe(
    vbt.Param(data, eval_id="outer"),
    vbt.Param(fast_windows, eval_id="inner", condition="fast_window < slow_window"),
    vbt.Param(slow_windows, eval_id="inner")
)
```

1. Inner decorator that iterates over fast and slow windows
2. Outer decorator that iterates over symbols
3. The same line as in the example above is now run for each symbol only - smart!
4. Call the inner function inside the outer function
5. Call the outer function on all parameters. For each parameter, specify which function it should be evaluated at.
6. Same function can be parameterized multiple times, where each decorator is responsible for evaluation of a subset of the passed parameters

Skipping

Parameter combinations can be skipped dynamically by returning **NoResult** instead of the actual result.

Skip the parameter combination if an error occurred

```
@vbt.parameterized
def my_pipeline(data, fast_window, slow_window):
    try:
        ...
        return result
    except Exception:
        return vbt.NoResult

results = my_pipeline(
    data,
    vbt.Param(fast_windows),
    vbt.Param(slow_windows)
)
```

Hybrid (mono-chunks)

The approach above calls the original function on each single parameter combination, which makes it slow when dealing with a large number of combinations, especially when each function call is associated with an overhead, such as when NumPy array gets converted to a Pandas object. Remember that 1 millisecond of an overhead translates into 17 minutes of additional execution time for one million of combinations.

There's nothing (apart from parallelization) we can do to speed up functions that take only one combination at a time. But if the function can be adapted to accept multiple combinations, where each parameter argument becomes an array instead of a single value, we can instruct `@vbt.parameterized` to merge all combinations into chunks and call the function on each chunk. This way, we can reduce the number of function calls significantly.

Test a grid of parameters using mono-chunks

```
@vbt.parameterized(mono_n_chunks=?, mono_chunk_len=?, mono_chunk_meta=?)  (1)
def my_pipeline(data, fast_windows, slow_windows):  (2)
    ...
    return result  (3)

results = my_pipeline(  (4)
    data,
    vbt.Param(fast_windows),
    vbt.Param(slow_windows)
)

# ------------------------------------------------------------

@vbt.parameterized(mono_n_chunks="auto")  (5)
...

# ------------------------------------------------------------

@vbt.parameterized(mono_chunk_len=100)  (6)
...
```

1. Instruct VBT to build chunks out of parameter combinations. You can use `mono_n_chunks` to specify the target number of chunks, or `mono_chunk_len` to specify the max number of combinations in each chunk, or `mono_chunk_meta` to specify the chunk metadata directly.
2. Function now must take multiple values `fast_windows` and `slow_windows` instead of single values `fast_window` and `slow_window`. One set of values will contain combinations that belong to the same chunk.
3. Do some calculations on the received parameter combinations and return a result (which should contain a result for each parameter combination)
4. Run the function the same way as before
5. Build the same number of chunks as there are CPU cores
6. Build chunks with at most 100 combinations each

By default, parameter values are passed as lists to the original function. To pass them as arrays or in any other format instead, set a merging function `mono_merge_func` for each parameter.

```
my_pipeline(
    param_a=vbt.Param(param_a),  (1)
    param_b=vbt.Param(param_b, mono_reduce=True),  (2)
    param_c=vbt.Param(param_c, mono_merge_func="concat"),  (3)
    param_d=vbt.Param(param_d, mono_merge_func="row_stack"),  (4)
```

```
    param_e=vbt.Param(param_e, mono_merge_func="column_stack"),  5
    param_f=vbt.Param(param_f, mono_merge_func=vbt.MergeFunc(...))  6
)
```

1. Will put chunk values into a list
2. Same as above but will return a single value if all values in the chunk are the same
3. Will concatenate values into a NumPy array or Pandas Series
4. Will stack chunk values along rows into a NumPy array or Pandas Series/DataFrame
5. Will stack chunk values along columns into a NumPy array or Pandas DataFrame
6. Will merge chunk values using a custom merging function

Execution is done in the same way as in **Parameterization** and chunks can be easily parallelized, just keep an eye on RAM consumption since now multiple parameter combinations are executed at the same time.

Example

See an example in **Mono-chunks**.

Chunking

Chunking revolves around splitting a value (such as an array) of one or more arguments into many parts (or chunks), calling the function on each part, and then merging all parts together. This way, we can instruct VBT to process only a subset of data at a time, which is helpful in both reducing RAM consumption and increasing performance by utilizing parallelization. Chunking is also highly convenient: usually, you don't have to change your function in any way, and you'll get the same results regardless of whether chunking was enabled or disabled. To use chunking, create a pipeline function, decorate it with `@vbt.chunked`, and specify how exactly arguments should be chunked and results should be merged.

Example

See an example in **Chunking**.

Decoration

To make any function chunkable, we have to decorate (or wrap) it with `@vbt.chunked`. This will return a new function with the same name and arguments as the original one. The only difference: this new function will process passed arguments, chunk the arguments, call the original function on each chunk of the arguments, and merge the results of all chunks.

Process only a subset of values at a time

```
@vbt.chunked
def my_pipeline(data, fast_windows, slow_windows):  1
    ...
    return result  2

results = my_pipeline(  3
    data,
    vbt.Chunked(fast_windows),  4
    vbt.Chunked(slow_windows)
)
```

1. Arguments can be anything. Here we're expecting a data instance, and already combined fast and slow windows, as in **Hybrid (mono-chunks)**
2. Do some calculations on the received chunk of values and return an result, which can be anything
3. Run the function the same way as without the decorator
4. Wrap any chunkable argument with `vbt.Chunked` or other class

To keep the original function separate from the decorated one, we can decorate it after it has been defined and give the decorated function another name.

Decorate a function later

```
def my_pipeline(data, fast_windows, slow_windows):
    ...
    return result

my_chunked_pipeline = vbt.chunked(my_pipeline)
results = my_chunked_pipeline(...)
```

Specification

To chunk an argument, we must provide a chunking specification for that argument. There are three main ways on how to provide such a specification.

Approach 1: Pass a dictionary `arg_take_spec` to the decorator. The most capable approach as it allows chunking of any nested objects of arbitrary depths, such as lists inside lists.

Specify chunking rules via arg_take_spec

```
@vbt.chunked(
    arg_take_spec=dict(  (1)
        array1=vbt.ChunkedArray(axis=1),  (2)
        array2=vbt.ChunkedArray(axis=1),
        combine_func=vbt.NotChunked  (3)
    ),
    size=vbt.ArraySizer(arg_query="array1", axis=1),  (4)
    merge_func="column_stack"  (5)
)
def combine_arrays(array1, array2, combine_func):
    return combine_func(array1, array2)

new_array = combine_arrays(array1, array2, np.add)
```

(1) Dictionary where keys are argument names and values are chunking rules for those arguments

(2) Split arguments `array1` and `array2` along columns. They must be multidimensional NumPy or Pandas arrays.

(3) Provide rules for all arguments. If any argument is missing in `arg_take_spec`, a warning will be thrown.

(4) Specify where the total size should be taken from. It's required to build chunks. This is mostly optional as newer versions of VBT can parse it automatically.

(5) Merging function must depend on the chunking arrays. Here, we should stack columns of output arrays back together.

Approach 2: Annotate the function. The most convenient approach as you can specify chunking rules next to their respective arguments directly in the function definition.

Specify chunking rules via annotations

```
@vbt.chunked
def combine_arrays(
    array1: vbt.ChunkedArray(axis=1) | vbt.ArraySizer(axis=1),  (1)
    array2: vbt.ChunkedArray(axis=1),
    combine_func
) -> "column_stack":
    return combine_func(array1, array2)

new_array = combine_arrays(array1, array2, np.add)
```

(1) Multiple VBT annotations can be combined with an `|` operator. Also, it doesn't matter whether a chunking annotation is provided as a class or an instance. Providing the sizer is mostly optional as newer versions of VBT can parse it automatically.

Approach 3: Wrap argument values directly. Allows switching chunking rules on the fly.

Specify chunking rules via argument values

```
@vbt.chunked
def combine_arrays(array1, array2, combine_func):
    return combine_func(array1, array2)

new_array = combine_arrays(  (1)
    vbt.ChunkedArray(array1),
    vbt.ChunkedArray(array2),
    np.add,
    _size=len(array1),  (2)
    _merge_func="concat"
)
new_array = combine_arrays(  (3)
    vbt.ChunkedArray(array1, axis=0),
    vbt.ChunkedArray(array2, axis=0),
    np.add,
    _size=array1.shape[0],
    _merge_func="row_stack"
)
new_array = combine_arrays(  (4)
    vbt.ChunkedArray(array1, axis=1),
    vbt.ChunkedArray(array2, axis=1),
    np.add,
    _size=array1.shape[1],
    _merge_func="column_stack"
)
```

(1) Split one-dimensional input arrays and concatenate output arrays back together

(2) Providing the total size is mostly optional as newer versions of VBT can parse it automatically

(3) Split two-dimensional input arrays along rows and stack rows of output arrays back together

(4) Split two-dimensional input arrays along columns and stack columns of output arrays back together

Merging and execution are done in the same way as in **Parameterization**.

Hybrid (super-chunks)

Parameterized decorator and chunked decorator can be combined to process only a subset of parameter combinations at a time without the need of changing the function's design as in **Hybrid (mono-chunks)**. Even though super-chunking may not be as fast as mono-chunking, it's still beneficiary when you want to process only a subset of parameter combinations at a time (but not all, otherwise, you should just use `distribute="chunks"` in the parameterized decorator without a chunked decorator) to keep RAM consumption in check, or when you want do some preprocessing and/or postprocessing such as flushing per bunch of parameter combinations.

Execute at most n parameter combinations per process

```
@vbt.parameterized
def my_pipeline(data, fast_window, slow_window):  1
    ...
    return result

@vbt.chunked(
    chunk_len=?,  2
    execute_kwargs=dict(chunk_len="auto", engine="pathos")  3
)
def chunked_pipeline(data, fast_windows, slow_windows):  4
    return my_pipeline(
        data,
        vbt.Param(fast_windows, level=0),
        vbt.Param(slow_windows, level=0)
    )

param_product = vbt.combine_params({  5
    "fast_windows": fast_windows,
    "slow_windows": slow_windows
}, build_index=False)

chunked_pipeline(
    data,
    vbt.Chunked(param_product["fast_windows"]),
    vbt.Chunked(param_product["slow_windows"])
)
```

1. Parameterized function expects a single parameter combination where each parameter argument is a single value
2. Split each sequence of parameter values into chunks of n elements each
3. Build super-chunks out of chunks where chunks within each super-chunk are executed in parallel and super-chunks themselves are executed serially
4. Chunked function expects a grid of parameter combinations where each parameter argument is a sequence of values. All sequences must have the same length.
5. Build a grid of parameter combinations

Raw execution

Whenever VBT needs to execute one function on multiple sets of arguments, it uses the function **execute**, which takes a list of tasks (functions and their arguments) and executes them with an engine selected by the user. This function takes all the same arguments that you usually pass inside `execute_kwargs`.

Execute multiple indicator configurations in parallel

```
sma_func = vbt.talib_func("sma")
ema_func = vbt.talib_func("ema")
tasks = [
    vbt.Task(sma_func, arr, 10),  1
    vbt.Task(sma_func, arr, 20),
    vbt.Task(ema_func, arr, 10),
    vbt.Task(ema_func, arr, 20),
]
keys = pd.MultiIndex.from_tuples([  2
    ("sma", 10),
    ("sma", 20),
    ("ema", 10),
    ("ema", 20),
], names=["indicator", "timeperiod"])

indicators_df = vbt.execute(  3
    tasks,
    keys=keys,
    merge_func="column_stack",
    engine="threadpool"
)
```

1. Each task consists of the function as well as the (positional and keyword) arguments it takes
2. Keys are displayed in the progress bar as well as in the columns of our new DataFrame
3. Execute tasks in separate threads and merge them into a DataFrame

———— + ————

If you want to parallelize a workflow within a for-loop, put it into a function and decorate that function with **iterated**. Then, when executing the decorated function, pass a total number of iterations or a range in place of the argument where you expect the iteration variable.

Execute a regular for-loop in parallel

```
# ------------------------------ FROM ------------------------------
```

```
results = []
keys = []
for timeperiod in range(20, 50, 5):
    result = sma_func(arr, timeperiod)
    results.append(result)
    keys.append(timeperiod)
keys = pd.Index(keys, name="timeperiod")
sma_df = pd.concat(map(pd.Series, results), axis=1, keys=keys)

# ____________________________ TO ____________________________

@vbt.iterated(over_arg="timeperiod", merge_func="column_stack", engine="threadpool")
def sma(arr, timeperiod):
    return sma_func(arr, timeperiod)

sma = vbt.iterated(  (1)
    sma_func,
    over_arg="timeperiod",
    engine="threadpool",
    merge_func="column_stack"
)

sma_df = sma(arr, range(20, 50, 5))
```

(1) Another way of decorating a function

Execute a nested for-loop in parallel

```
# ____________________________ FROM ____________________________

results = []
keys = []
for fast_window in range(20, 50, 5):
    for slow_window in range(20, 50, 5):
        if fast_window < slow_window:
            fast_sma = sma_func(arr, fast_window)
            slow_sma = sma_func(arr, slow_window)
            result = fast_sma - slow_sma
            results.append(result)
            keys.append((fast_window, slow_window))
keys = pd.MultiIndex.from_tuples(keys, names=["fast_window", "slow_window"])
sma_diff_df = pd.concat(map(pd.Series, results), axis=1, keys=keys)

# ____________________________ TO ____________________________

@vbt.iterated(over_arg="fast_window", merge_func="column_stack", engine="pathos")  (1)
@vbt.iterated(over_arg="slow_window", merge_func="column_stack", raise_no_results=False)
def sma_diff(arr, fast_window, slow_window):
    if fast_window >= slow_window:
        return vbt.NoResult
    fast_sma = sma_func(arr, fast_window)
    slow_sma = sma_func(arr, slow_window)
    return fast_sma - slow_sma

sma_diff_df = sma_diff(arr, range(20, 50, 5), range(20, 50, 5))
```

(1) Execute the outer loop in parallel using multiprocessing